

# PM AI Bootcamp — Capstone Playbook: From Brief to Customer Zero v2.0

**When to read:** immediately after a team receives one of the six project briefs and enters **Phase 3 - Build your Customer Zero**.

**Problem solved by this file:** "We got the brief. What do we do first, and in what order?"

**Core principle:** humans decide, AI executes; build the **foundation before the surface**, build a **thin vertical slice before broad coverage**, and log Dev Book + telemetry while the work is happening.

## Rule zero — AI-generated does NOT mean done

This playbook lets Claude Code generate a large amount of output very quickly. That speed is useful, but it is also the trap.

**Done** does not mean:

- "AI returned a file"
- "the artifact looks polished"
- "the structure sounds plausible"

**Done** means:

1. you can explain the step in your own words,
2. you can point to at least one AI mistake, omission, or weak assumption and correct it,
3. the relevant gate is truly passed.

Every step below therefore has a **Understanding Gate**. If the gate fails, the step is not complete.

## One-page map — 11 steps from brief to Customer Zero

```
Brief (PB-0X)
->
[0] Clarify scope
->
```

```
[1] Detailed SW spec / SRS
  ->
[2] Module map and layering
  ->
[3] Architecture
  ->
[4] WBS + rolling-wave plan
  ->
[5] Estimation
  ->
[6] Risk + governance + delegation map
  ->
[7] Definition of Ready for the first vertical slice
  ->
[8] Build the vertical slice
  ->
[9] Test and gate
  ->
[10] Traceability + telemetry + weekly report
  ->
Viva
```

Suggested artifact names:

- SCOPE-PB0X.md
- SPEC-PB0X.md
- MODULEMAP-PB0X.md
- ARCH-PB0X.md
- WBS-PB0X.md
- EST-PB0X.md
- RISK-PB0X.md
- DELEGATION-MAP-PB0X.md
- DOR-PB0X.md
- DEVB00K-PB0X.md
- RTM-PB0X.md
- WEEKLY-PB0X.md

---

## Standard orchestration loop for every step

At every stage, use the same rhythm:

Context -> Plan -> Delegate -> Execute -> Gate -> Log -> Iterate

This playbook assumes Claude Code is acting as the orchestrator. The learner is responsible for:

- approving the plan,
- choosing the leash,
- keeping the gates honest,
- logging corrections and evidence.

---

## [0] Receive the brief and clarify scope

**Goal:** turn the brief into a defensible MVP boundary.

**Inputs:** project brief + clarification prompts from the brief.

**Output:** SCOPE-PB0X.md

**Done when:** the team can name what is in scope, what is explicitly out of scope, which assumptions are carrying risk, and which architectural decisions depend on them.

**Understanding Gate:** explain one assumption the AI made that you rejected or rewrote.

---

## [1] Detailed SW spec / SRS

**Goal:** create a specification strong enough for design and testing.

**Output:** SPEC-PB0X.md

Include:

- functional requirements,
- user stories,
- acceptance criteria,
- role-based use cases,
- key NFRs.

**Understanding Gate:** walk through one story and say what would count as PASS and FAIL.

---

## [2] Module map and layering

**Goal:** separate foundation work from surface features.

**Output:** MODULEMAP-PB0X.md

The key decision is always:

- what is **Layer 0 / hidden foundation**,
- what is **surface module**,
- what is the first **thin vertical slice** that proves the path.

**Understanding Gate:** explain why one foundation piece must exist before a visible feature can work.

---

## [3] Architecture

**Goal:** design enough structure to begin real build work.

**Output:** ARCH-PB0X.md

Expected content:

- architecture principles,
- container / system view,
- data model,
- API contracts,
- technology choices,
- permission model,
- unresolved decisions.

**Understanding Gate:** justify one architecture choice and name one trade-off.

---

## [4] WBS + rolling-wave plan

**Goal:** break the build into work at the right level of detail.

**Output:** WBS-PB0X.md

Rules:

- near-term wave = task-level,
- next wave = feature-level,
- later waves = epic-level.

**Understanding Gate:** point out one commonly missed task that AI did not include at first.

---

## [5] Estimation

**Goal:** estimate based on assumptions, not on wishful thinking.

**Output:** EST-PB0X.md

Use:

- three-point estimation,
- explicit assumptions,
- risk-aware buffers,
- a realism check against the project timeline.

**Understanding Gate:** explain one estimate and the assumption behind it.

---

## [6] Risk + governance + delegation map

**Goal:** make risk ownership and AI boundaries explicit.

**Outputs:**

- RISK-PB0X.md
- DELEGATION-MAP-PB0X.md

Required elements:

- technical and schedule risks,
- AI-specific risks,
- owner and mitigation,
- L0-L5 assignment,
- leash A / A+,
- fail-closed gates.

**Understanding Gate:** explain why one task must be A+ instead of A.

---

## [7] Definition of Ready for the first vertical slice

**Goal:** prove the team is actually ready to build the first slice.

**Output:** DOR-PB0X.md

Check:

- spec clarity,
- data model,
- API contract,
- acceptance criteria,
- governance level,
- gates.

**Understanding Gate:** point to one DoR item that is still red and explain why build must wait.

---

## [8] Build the vertical slice

**Goal:** produce a working, runnable slice of the real system. **This is graduation evidence number one.**

**Outputs:**

- working code / runnable slice,
- DEVB00K-PB0X.md

The Dev Book must record:

- where AI was wrong,
- what the learner corrected,
- which autonomy level was used,
- which gate blocked risk,
- where hard-stop happened.

**Understanding Gate:** after each meaningful build step, explain what changed and record at least one real AI correction. A Dev Book with zero AI corrections means rubber-stamping is likely and the step is not considered complete.

## Two capstone tiers — choose by technical background

The program's goal is for PMs to **orchestrate AI delivery**, not to turn PMs into developers. Both tiers require **end-to-end system understanding**, **authentic Dev Book entries**, and the ability to **demo a running flow**. The difference is technical depth.

| Aspect                | Tier 1 — Full-stack                                                 | Tier 2 — PM-stack                                                                                                                                                             |
|-----------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Who</b>            | PM with technical background or prior coding experience             | Non-coder PM, pure management background                                                                                                                                      |
| <b>Code</b>           | Real running code (Next.js / React)                                 | Interactive prototype in the browser (HTML/CSS/JS or Next.js static) — click a button, navigate pages, display data                                                           |
| <b>Backend</b>        | Real running API (NestJS / Spring Boot)                             | API mock (json-server or mock endpoint). PM <b>defines the correct API contract</b> (endpoint, input, output, error codes)                                                    |
| <b>Database</b>       | Real PostgreSQL + migrations                                        | SQL schema written correctly (PM drives AI to generate it, reviews table structure, checks foreign-key constraints, tags sensitive fields) — no need to run actual migrations |
| <b>Tests</b>          | Automated unit + integration tests pass                             | PM writes <b>test scenarios</b> in text (Given/When/Then), AI generates test scripts, PM runs them and reads pass/fail results — no need to debug test code                   |
| <b>Dev Book focus</b> | Where <b>AI generated wrong code</b> (logic, security, performance) | Where <b>AI made design-level mistakes</b> (wrong API contract fields, missing schema columns, UI flows that do not match acceptance criteria)                                |
| <b>Demo</b>           | End-to-end flow running live on browser + DB                        | End-to-end flow running on browser (prototype + mock data)                                                                                                                    |

**Choosing a tier:** learners self-select and the coach confirms at the start of Phase 3.

**Choosing Tier 2 carries no penalty** — both tiers qualify for **Certified**. Tier 1 has an advantage when evaluating for Distinction/Champion (deeper evidence), but Tier 2 **does not block graduation**.

**Why Tier 2 exists:** PMs need to understand data models, API contracts, and test scenarios — that is essential architecture knowledge. But forcing a PM to debug NestJS runtime errors leads to either giving up or rubber-stamping code they do not understand — the opposite of what the program teaches. Tier 2 keeps the parts a PM **must** understand and removes the parts a PM **does not need** to do.

---

## [9] Test and gate

**Goal:** prove the slice works through gates, not vibes.

**Expected evidence:**

- test results,
- negative cases,
- role / permission checks,
- gate outcomes.

**Rule:** if it cannot be verified, it does not pass.

**Understanding Gate:** show one failed or risky case that would have slipped through without the gate.

---

## [10] Traceability + telemetry + weekly report

**Goal:** make the slice explainable, measurable, and reviewable.

**Outputs:**

- RTM-PB0X.md
- telemetry
- WEEKLY-PB0X.md

Telemetry should include:

- real human hours,
- correction loops,
- rework,
- token estimate / reconciliation,
- quality or defect observations.

**Understanding Gate:** explain one KPI and where its source data comes from.

---

## Viva preparation

The capstone is not complete until the learner can defend it live.

Graduation still depends on the **three hard gates** (fail any one and the capstone does not pass):

1. **Capability score** across the six pillars meets the threshold (no pillar at zero).
2. **Working system** — live demo of the vertical slice.
3. **Viva pass** — answer tough questions: *why this architecture? where did AI fail and how did you fix it? if constraint X changes, what breaks?*

#### Tier-specific viva questions:

- **Tier 1:** *"Show us the running system. Walk through one end-to-end flow. Why does the code handle edge case X this way? Where did AI generate wrong code and how did you debug it? If the DB schema changes, which APIs break?"*
- **Tier 2:** *"Show your API mock. Demonstrate one test scenario. Explain what would change in production. Why does this API need field X? If a new role is added, where does the schema change? Where did AI produce a wrong API contract and how did you fix it?"* — the coach tests **systems thinking**, not coding skill.

**Portfolio for the viva:** SPEC , ARCH , WBS , RISK , DELEGATION-MAP , DOR , DEVB00K , RTM , telemetry, WEEKLY + working system (Tier 1: running code + passing tests; Tier 2: prototype + API contract + schema + test scenarios). The coach scores **your judgment**, not how polished the AI output looks.

**What will the coach ask?** Questions come straight from the **Understanding Gates** in each step: point to any line in any artifact and ask *"why?"* or *"where did AI get this wrong and how did you fix it?"*. A team that truly passed the gates along the way will answer naturally; a team with a stack of pretty files but no understanding will fail at the first question.

## Common failure modes

- letting AI generate many artifacts but not understanding any of them,
- skipping gates because the output "looks fine",
- building a wide feature set before a thin vertical slice,
- failing to record Dev Book evidence,
- estimating without assumptions,
- over-delegating high-risk tasks.

The playbook exists to keep the capstone grounded in **understanding, evidence, and real execution**, not document volume.

## Appendix — Glossary



For terminology lookup, see [docs/GLOSSARY.md](#) — the unified glossary for the entire program, with a column mapping to international industry terms.

**Related documents:** *Project Briefs* (6 briefs) · *Workbook Common* (EX-01 through EX-06) + role-specific Workbooks for BA/SA/Dev · *Orchestrator Guide* · *Program sections 7-10* (assignment, rubric, KPI) · *Operating Model* (L0-L5, Leash, governance).

## Appendix — Changelog

| Version | Date       | Changes                                                                                                                                                                                                                                                                                                                                                                                               |
|---------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v2.0    | 2026-08-24 | <b>Added 2-tier Capstone:</b> step [8] now offers Tier 1 (full-stack, real running code) and Tier 2 (PM-stack: prototype + API mock + schema + test scenarios) for non-coder PMs. Viva updated with tier-specific questions. Inline glossary replaced with pointer to <a href="#">docs/GLOSSARY.md</a> . Changelog moved to end.                                                                      |
| v1.1    | 2026-07-09 | Added <b>anti-rubber-stamp mechanism</b> : "Rule zero — AI-generated does NOT mean done" callout at top + <b>Understanding Gates</b> at all 11 steps (each gate requires <i>explaining in your own words</i> + <i>catching at least one AI mistake</i> ). Patches the risk of "generating fast but understanding nothing".                                                                            |
| v1.0    | 2026-07-09 | Initial version: 11-step pipeline from brief to working system to viva; each step has goal, input, orchestration guidance, output, common traps, done criteria, with <b>"What RevenueOS did"</b> reference boxes pointing to real artifacts (PRD, Use-Case, Module Map, Architecture, Tech-Spec, Work Breakdown, DoR L4, RTM, SIT/UAT, Telemetry). Maps to the 6 Workbook drills EX-01 through EX-06. |